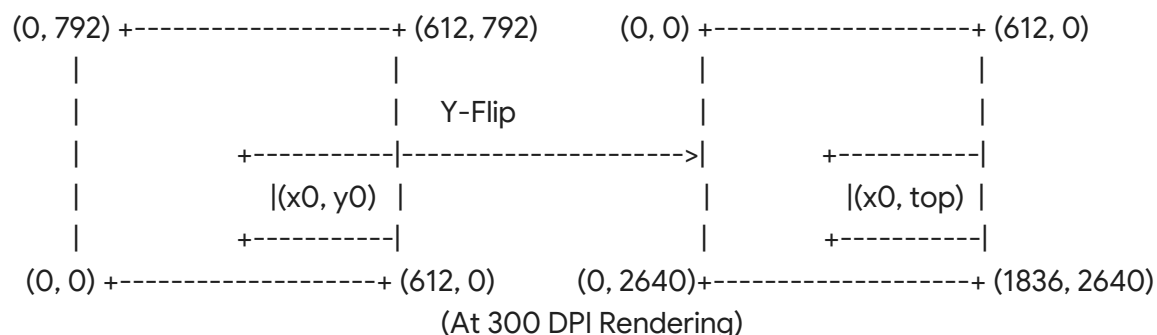




Transformation Formula & DPI Scaling

When converting native PDF coordinates to an image overlay or OCR bounding box at a target resolution (e.g., 300 DPI):

Scale Factor = Target DPI / 72 (e.g., $300 / 72 = 4.1667$)

$x_{\text{pixel}} = x_{\text{pdf}} * \text{Scale Factor}$

$y_{\text{pixel}} = (\text{Page Height}_{\text{pdf}} - y1_{\text{pdf}}) * \text{Scale Factor}$

2. Bounding Box Formats

In document layout parsing, bounding boxes are represented in three standard formats:

- **[x0, y0, x1, y1] (Corners):**
 - (x0, y0) represents the bottom-left (PDF) or top-left (Image) corner.
 - (x1, y1) represents the top-right (PDF) or bottom-right (Image) corner.
- **[x, y, width, height] (Dimensions):**
 - x, y is the anchor point, accompanied by explicitly calculated width = $x1 - x0$ and height = $y1 - y0$.
- **Quadrilateral Polygons [(x1,y1), (x2,y2), (x3,y3), (x4,y4)]:**
 - Required when text is rotated or skewed (e.g., a diagonal watermark or rotated sidebar text) where a simple axis-aligned rectangle would cover non-text regions.

3. Granular Levels of Extraction

Bounding boxes exist at four distinct hierarchical levels:

Document Level

└ Page Level

└ Block Bounding Box (Paragraphs / Sections / Tables)

└ Line Bounding Box (Horizontal alignment baseline)

└ Word Bounding Box (Grouped glyphs separated by space thresholds)

└ Character/Glyph Bounding Box (Exact vector bounds per character)

- **Character/Glyph Level:** The exact vector bounding rectangle for each individual glyph. Crucial for detecting precise spacing, subscripts, superscripts, or mathematical formulas.
- **Word Level:** Formed by merging character boxes based on horizontal proximity thresholds (e.g., if space between character i and $i+1$ exceeds $0.2 * \text{font_size}$, a word boundary is inserted).
- **Line Level:** Groups words sharing a common vertical baseline $y0 = y1$.
- **Block Level:** Combines lines based on vertical line spacing (leading) and horizontal alignment boundaries.

4. Extraction Mechanics in Popular Libraries

Different Python open-source tools handle coordinates differently:

PyMuPDF (fitz):

- Uses top-left relative float coordinates: `Rect(x0, y0, x1, y1)`.
- `page.get_text("words")` returns tuples: `(x0, y0, x1, y1, "word_text", block_no, line_no, word_no)`.
- `page.get_text("blocks")` returns block-level rectangles with text and bounding boxes.

pdfplumber:

- Exposes explicit properties per character object: `x0`, `top`, `x1`, `bottom`, `doctop`, `width`, `height`, `fontname`, `size`.
- Uses top-left as `(0, 0)` (`top` measures distance from the top of the page).

pdfminer.six:

- Exposes lower-level PDF native coordinates (bottom-left origin).
- Returns `LTChar`, `LTTextBox`, `LTTextLine`, allowing raw geometry inspection.

5. Edge Cases & Risks in Coordinate Extraction

- **CropBox vs. MediaBox Mismatches:**
 - A PDF page might have a `MediaBox` of `[ ]`, but a `CropBox` of `[ ]`.
 - If a parser ignores the `CropBox` origin offset, all bounding box calculations will be shifted by `(+50, +50)` points.
- **Transform Matrices & Rotation:**
 - Text can be placed with a Current Transformation Matrix (CTM): `[a b 0, c d 0, e f 1]`
 - If text is rotated 90 degrees or 270 degrees, raw `x0`, `y0`, `x1`, `y1` values become inverted, causing spatial clustering algorithms to miscalculate line heights and column widths.
- **Subpixel Overlaps & Floating Point Drift:**
 - Due to rounding in PDF generators, two characters on the exact same line might have slightly different baseline y-coordinates (e.g., `y1 = 120.002` vs `y2 = 120.045`). Hard equality checks `y1 == y2` fail; tolerance bands (`+/- 1.5 pt`) must be used.